

How Do Positional Encodings Affect Transformer Training in Low-Resource Settings?

Vansh Tiwari

vanshtiwari1512@gmail.com

Abstract

Transformers rely on positional encodings to model token order, yet most prior studies evaluate positional encoding strategies under large-scale training settings. In this work, we investigate how different positional encodings affect Transformer optimization behavior when training data is severely limited. We implement a standard encoder–decoder Transformer from scratch and train it on a low-resource German–English translation task while varying only the positional encoding mechanism. Specifically, we compare fixed sinusoidal positional encodings, learned absolute positional embeddings, and rotary positional encodings under identical architectures, hyperparameters, and data budgets. Due to instability of generation-based metrics in this regime, we evaluate models using training and validation cross-entropy loss. Our results show that learned positional embeddings provide no advantage over fixed sinusoidal encodings, while rotary positional encoding results in slower convergence and higher validation loss. These findings suggest that under extreme low-resource and early-training conditions, positional encoding choice has limited impact on Transformer optimization, and simpler encoding schemes are sufficient.

1 Introduction

Transformer architectures have become the dominant modeling paradigm in natural language processing, enabling strong performance across tasks such as machine translation, language modeling, and text understanding [3]. A key component of the Transformer model is positional encoding, which injects information about token order into an otherwise permutation-invariant self-attention mechanism. The original Transformer introduced fixed sinusoidal positional encodings, while subsequent work has proposed alternatives such as learned absolute embeddings and relative or rotary positional encodings [1].

Most existing studies evaluating positional encoding strategies focus on large-scale training regimes, where models are trained on millions of examples for many epochs. In such settings, differences between positional encoding schemes are often analyzed in terms of final task performance or long-context generalization. However, comparatively little attention has been paid to understanding how positional encodings affect optimization behavior when Transformers are trained under severe data constraints.

Low-resource training scenarios are increasingly relevant in practice, particularly for languages, domains, or tasks where annotated data is scarce. In these regimes, models are often trained from scratch using limited datasets and few training epochs, making early optimization dynamics a critical factor. It remains unclear whether more expressive positional encoding mechanisms provide meaningful benefits under such constraints or whether simpler encodings suffice.

In this work, we conduct a controlled empirical study to examine how different positional encoding strategies influence the training behavior of Transformers in an extremely low-resource setting. We implement a standard encoder–decoder Transformer from scratch and train it on a small German–English translation dataset derived from OPUS [2], while varying only the positional encoding mechanism. Specifically, we compare fixed sinusoidal positional encodings, learned absolute positional embeddings, and rotary positional encodings under identical model architectures, training hyperparameters, and data budgets.

Rather than focusing on final translation quality, which can be unstable for scratch-trained models in low-data regimes, we evaluate performance using training and validation cross-entropy loss. This allows us to analyze convergence speed and generalization behavior during early training. Our results show that learned positional embeddings offer no observable advantage over fixed sinusoidal encodings in this setting, while rotary positional encodings lead to slower convergence and higher validation loss.

The contributions of this paper are threefold: (1) we provide a clean, reproducible analysis of positional encoding behavior under extreme data limitations; (2) we demonstrate that increased positional encoding complexity does not necessarily improve optimization in low-resource training; and (3) we release our implementation and experimental configurations to facilitate further investigation into positional encoding design under constrained regimes.

2 Methodology

2.1 Transformer Architecture

We employ a standard encoder–decoder Transformer architecture as originally proposed by Vaswani et al. [3]. The model consists of stacked self-attention and feedforward layers in both the encoder and decoder. Self-attention enables each token to attend to all other tokens in the sequence, allowing the model to capture long-range dependencies without recurrent or convolutional components.

All experiments use an identical Transformer configuration. The model dimensionality is set to 512, with 8 attention heads and 2048-dimensional feedforward layers. Both the encoder and decoder contain four layers. Dropout is applied with a rate of 0.1 throughout the model. Token embeddings are learned and shared across all experiments.

Since self-attention is permutation-invariant, the Transformer requires an explicit mechanism to encode token order. This is achieved through positional encoding, which is added to token embeddings before they are passed into the encoder and decoder.

2.2 Positional Encoding Variants

We compare three positional encoding strategies that differ in how positional information is represented and incorporated into the model. Importantly, all comparisons are conducted under identical architectural and training settings, with positional encoding as the only variable.

2.2.1 Sinusoidal Positional Encoding

The sinusoidal positional encoding is a fixed, non-trainable method that assigns each position a deterministic vector based on sine and cosine functions of varying frequencies. This approach enables the model to generalize to sequence lengths not observed during training. Following common practice, the positional encoding vector is added element-wise to the token embedding at each position. Since these encodings are fixed, they do not introduce additional trainable parameters into the model.

2.2.2 Learned Positional Embedding

In the learned positional embedding approach, each position is associated with a trainable embedding vector of the same dimensionality as the token embedding. These positional embeddings are initialized randomly and learned jointly with the rest of the model parameters through backpropagation. Unlike sinusoidal encodings, learned embeddings increase the number of trainable parameters and may adapt more flexibly to the training data, particularly when positional patterns are consistent.

2.2.3 Rotary Positional Encoding

Rotary positional encoding introduces relative position information directly within the self-attention mechanism by rotating the query and key vectors as a function of their position [1]. This method encodes position through multiplicative interactions rather than additive embeddings, enabling the attention scores to depend explicitly on relative token offsets. Rotary encodings do not require explicit positional embeddings to be added to the input representations.

2.3 Experimental Control

To ensure a fair comparison, we control all aspects of the model architecture, optimization procedure, and dataset across experiments. The same Transformer implementation, hyperparameters, optimizer, and data splits are used for all positional encoding variants. As a result, observed differences in training and validation behavior can be attributed solely to the choice of positional encoding.

3 Experimental Setup

3.1 Dataset and Low-Resource Setting

All experiments are conducted on the OPUS Books dataset for German–English machine translation [2]. To simulate an extremely low-resource scenario, we restrict the training set to a subset of 2,000 sentence pairs. A fixed validation split is used to monitor generalization during training. Test-set evaluation is not performed due to the instability of generation-based metrics when training Transformers from scratch with limited data.

We tokenize input sentences using the pretrained Marian tokenizer (`Helsinki-NLP/opus-mt-de-en`). Token sequences are truncated or padded to a maximum length of 64 tokens. The same vocabulary and preprocessing pipeline are used across all positional encoding variants.

3.2 Model Configuration

All experiments use the same Transformer encoder–decoder architecture described in Section 2. The model dimensionality is set to 512 with 8 attention heads and four layers in both the encoder and decoder. Feedforward layers use a hidden dimension of 2048, and dropout is applied at a rate of 0.1.

To ensure a controlled comparison, all model parameters except for the positional encoding mechanism are identical across experiments.

3.3 Training Procedure

Models are trained from scratch for three epochs using the AdamW optimizer with a learning rate of 3×10^{-4} and a batch size of 32. Cross-entropy loss is used as the training objective, with padding tokens ignored during loss computation. All experiments are conducted on a single GPU when available.

Given the limited dataset size and short training horizon, optimization behavior during early training is the primary focus of evaluation.

3.4 Evaluation Strategy

We evaluate models using training and validation cross-entropy loss rather than sequence-level metrics such as BLEU. Preliminary experiments showed that BLEU scores were unstable and frequently degenerate for scratch-trained models in this low-resource regime.

Tracking loss curves over epochs allows us to directly compare convergence speed and generalization behavior across positional encoding strategies. Loss values are recorded after each epoch for all experiments.

3.5 Compared Positional Encodings

We compare three positional encoding strategies under identical training conditions:

- **Sinusoidal Positional Encoding:** fixed, non-trainable embeddings added to token representations.
- **Learned Positional Embedding:** trainable absolute positional vectors learned jointly with the model.
- **Rotary Positional Encoding:** relative positional encoding applied within the self-attention mechanism.

Each experiment differs only in the positional encoding mechanism, ensuring that observed differences are attributable solely to positional encoding choice.

4 Results and Analysis

4.1 Training and Validation Loss

Table 1 summarizes the final training and validation loss values obtained for each positional encoding strategy after three epochs of training in the low-resource setting. All models are trained with identical architectures, hyperparameters, and data splits.

Positional Encoding	Train Loss	Validation Loss
Sinusoidal	5.8132	5.8166
Learned	5.8284	5.8113
Rotary	5.8420	5.8576

Table 1: Final training and validation loss after three epochs.

The sinusoidal and learned positional encoding variants exhibit nearly identical training and validation loss values, while the rotary positional encoding results in slightly higher loss on both splits.

4.2 Convergence Behavior

Figure 1 illustrates the training and validation loss curves over epochs for all three positional encoding strategies. Under the extreme low-resource constraint, both sinusoidal and learned positional encodings demonstrate similar convergence behavior, with steady loss reduction and no evidence of overfitting.

In contrast, rotary positional encoding converges more slowly and achieves higher validation loss throughout training. This suggests that relative positional encoding may introduce additional optimization difficulty when data is scarce and training duration is limited.

4.3 Effect of Positional Encoding Choice

Across all experiments, we observe no measurable advantage of learned positional embeddings over fixed sinusoidal encodings in the low-resource regime considered. Despite their additional trainable parameters, learned positional embeddings do not improve convergence speed or validation performance.

The inferior performance of rotary positional encoding in this setting indicates that mechanisms designed to capture relative positional information may require larger datasets or longer training horizons to be effective.

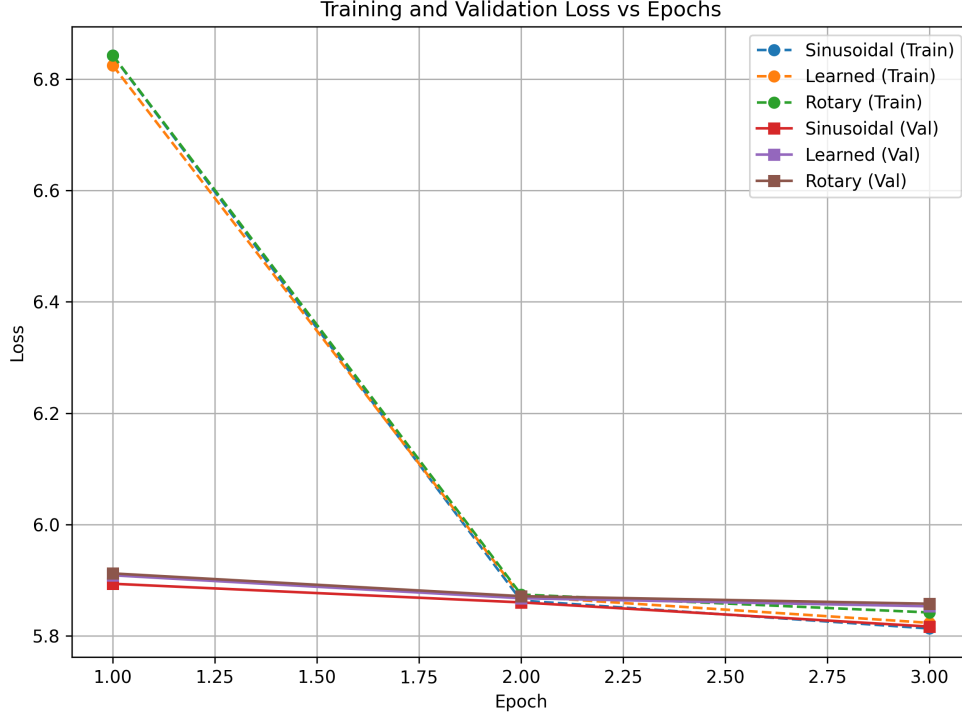


Figure 1: Training and validation loss curves for different positional encoding strategies under low-resource training.

When training data is severely limited, simpler absolute positional encodings appear sufficient for early-stage optimization.

4.4 Summary of Findings

These results indicate that, under extreme data constraints, the choice of positional encoding has limited impact on early Transformer training. Increasing the complexity of the positional encoding mechanism does not necessarily yield optimization benefits and may, in some cases, hinder convergence.

5 Discussion and Limitations

The results presented in Section 4 provide insight into the role of positional encoding during early Transformer training under extreme data constraints. Across all experiments, fixed sinusoidal and learned absolute positional encodings exhibit nearly identical optimization behavior, while rotary positional encoding consistently converges more slowly and achieves higher validation loss.

A possible explanation for the lack of improvement observed with learned positional embeddings lies in the limited size of the training data. With only 2,000 training examples and three training epochs, the model may not receive sufficient signal to meaningfully specialize additional positional parameters. Under such conditions, the fixed structure imposed by sinusoidal encodings may act as an implicit regularizer, preventing the model from overfitting positional patterns in the data.

The degraded performance of rotary positional encoding in this regime suggests that relative positional representations may be more sensitive to data scarcity. While rotary encodings have demonstrated effective-

ness in large-scale and long-context settings [1], they may require longer training horizons or larger datasets to realize their advantages. In low-resource scenarios, the additional complexity introduced into the attention mechanism may hinder stable optimization.

It is important to emphasize that these findings are specific to the experimental conditions explored in this work. The results should not be interpreted as a general claim that positional encoding choice is inconsequential for Transformers. Rather, they indicate that under extreme low-resource and early-training conditions, simpler absolute positional encodings are sufficient for optimization, and more expressive alternatives may not provide immediate benefits.

5.1 Limitations

This study has several limitations. First, experiments are conducted on a single language pair and a single dataset, which may limit generalizability to other tasks or domains. Second, models are trained for only three epochs due to the focus on early optimization behavior; longer training schedules may yield different trends. Third, evaluation relies exclusively on cross-entropy loss, as generation-based metrics such as BLEU were found to be unstable in this regime. Finally, the experiments do not explore hybrid or adaptive positional encoding strategies, which may offer improvements under constrained data settings.

6 Conclusion

In this work, we studied the effect of different positional encoding strategies on Transformer training behavior under severe data limitations. By training a standard Transformer model from scratch on a low-resource German–English translation task, we conducted a controlled comparison of sinusoidal, learned absolute, and rotary positional encodings while holding all other factors constant.

Our experiments show that learned positional embeddings do not provide an advantage over fixed sinusoidal encodings in this regime, despite introducing additional trainable parameters. Furthermore, rotary positional encoding leads to slower convergence and higher validation loss, suggesting that relative positional representations may be less effective during early optimization when data is scarce.

These findings indicate that, under extreme low-resource and early-training conditions, the choice of positional encoding has limited impact on Transformer optimization, and simpler encoding schemes are sufficient. More expressive positional encoding methods may require larger datasets or longer training horizons to realize their benefits.

Future work could extend this analysis to additional datasets, longer training schedules, and alternative low-resource tasks. Investigating hybrid or adaptive positional encoding strategies may further improve Transformer performance under data-constrained settings.

References

- [1] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [2] Jörg Tiedemann and Santhosh Thottingal. Opus-*mt* – building open translation services for the world. In *Proceedings of the EAMT*, 2020.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.